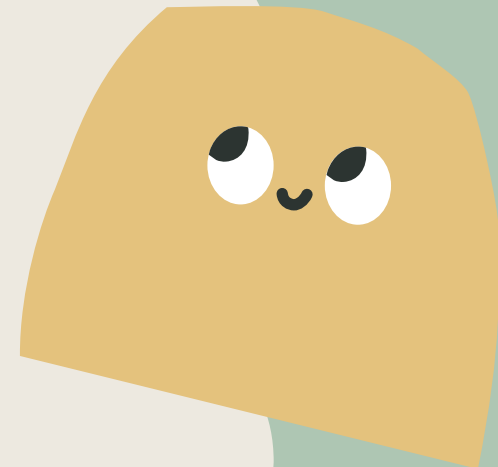
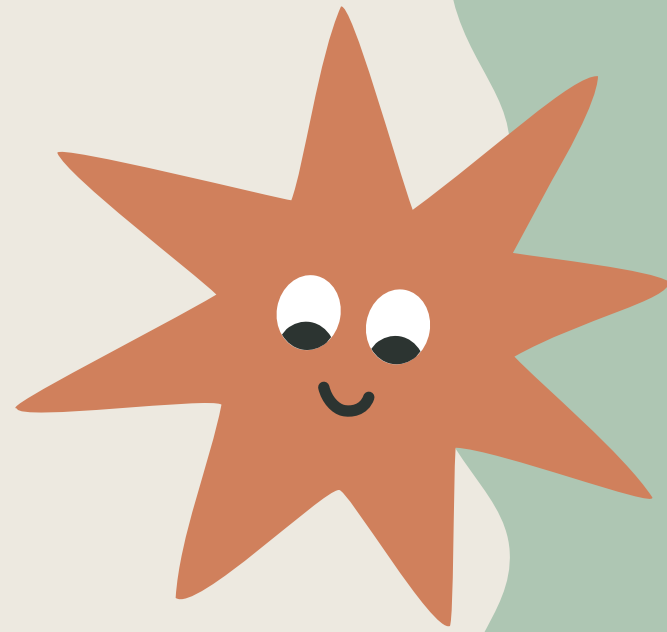


Array & Array Operations – Traversal, Insertion, Deletion, Searching & Updating

Lecture 6

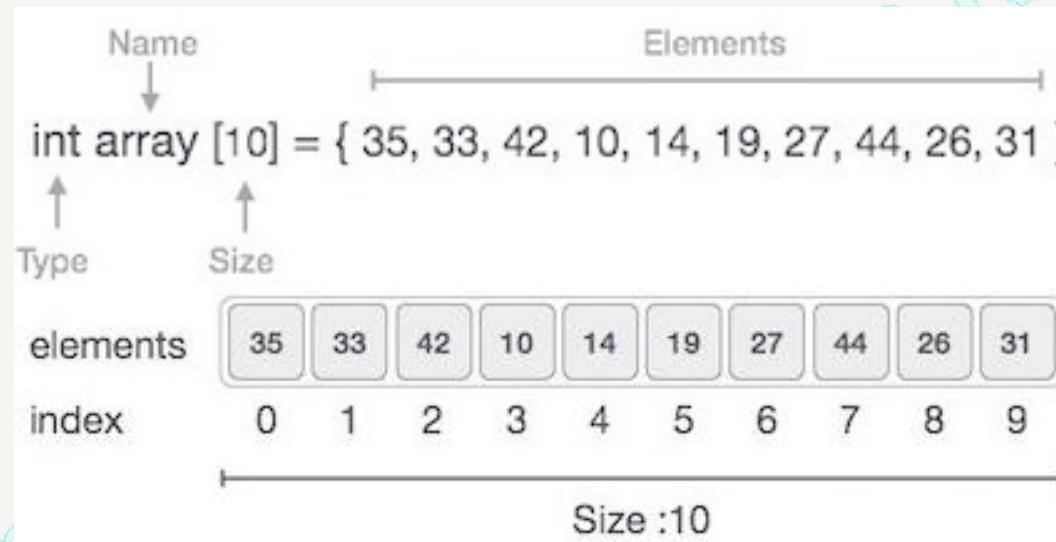


Learning Outcomes (Lecture 6)

- At the end of this lecture, students will be able to:
 - Implement array traversal.
 - Insert elements at the beginning, end, and any position.
 - Delete elements from the beginning, end, and any position.
 - Search an element using Linear Search.
 - Update an existing element.
 - Analyze the time complexity of each operation.

Array

- An array is a collection of elements of the same data type stored in contiguous memory locations and accessed using an index.



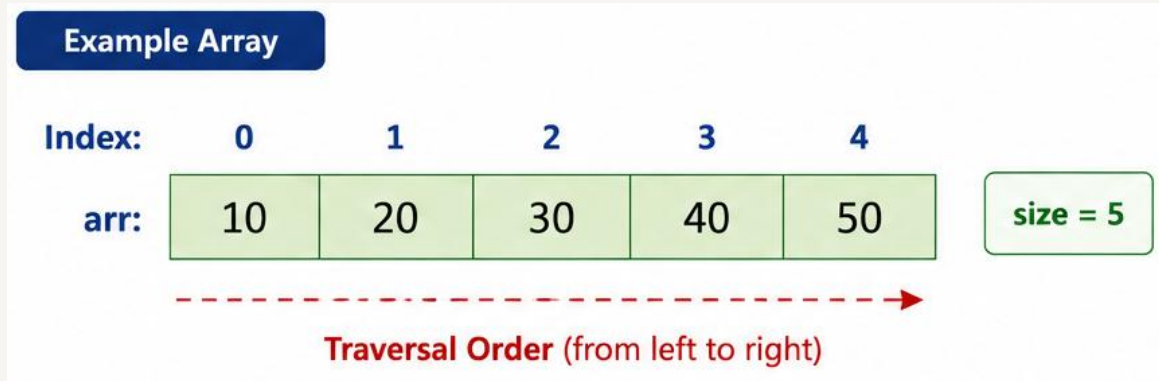
- Accessing an array element is random e.g. **`cout<<array[2]`**
- Accessing an array element takes $O(1)$ time

Basic Operations of an Array

- Following are the basic operations supported by an array:
- **Traverse** – Print all the array elements one by one
 - **Insertion (3 Cases)** – Adds an element at the given index, adds at start, add at end of array
 - **Deletion (3 Cases)** – Deletes an element at the given index, delete from start, delete at the end of array
 - **Search** – Searches an element using the given index or by the value
 - **Update** – Updates an element at the given index

Traverse

- Traversal means visiting each element of an array exactly once in a sequential order from the first element to the last element



Time Complexity = $O(n)$

C++ Implementation

```
void traverse(int arr[], int size)
{
    for(int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

Insertion – Insert at Beginning

Problem Statement

Insert a new element at the beginning of the array.

Example: Insert **5** at the beginning.

Initial Array

Index:	0	1	2	3	4
Value:	10	20	30	40	50

size = 5, capacity = 10

After Insertion (Insert 5 at Beginning)

Index:	0	1	2	3	4	5
Value:	5	10	20	30	40	50

size = 6, capacity = 10

How it works?

Step 1: Shift all elements one position to the right starting from the last element.

Index:	0	1	2	3	4	5
Value:	10	20	30	40	50	

Step 2: Place the new element at index 0.

Index:	0	1	2	3	4	5
Value:	5	10	20	30	40	50

Insertion – Insert at Beginning (C++ Code)

C++ Code

```
// Insert at beginning
void insertAtBeginning(int arr[], int &size,
    int capacity, int value)
{
    if (size == capacity)
    {
        cout << "Array is Full!\n";
        return;
    }
    // Step 1: Shift elements to the right
    for (int i = size; i > 0; i--)
        arr[i] = arr[i - 1];

    // Step 2: Insert new element at index 0
    arr[0] = value;
    size++;
}
```

Time Complexity: $O(n)$

Insertion – Insert at End

Example

Initial Array

Index:	0	1	2	3	4	5	6	7	8	9
Value:	10	20	30	40	50					



We will insert 60 at the end.

Step 1: Place new element at index = size (5)

Index:	0	1	2	3	4	5	6	7	8	9
Value:	10	20	30	40	50	60				



Step 2: Increase size by 1

Index:	0	1	2	3	4	5	6	7	8	9
Value:	10	20	30	40	50	60				



Time Complexity: $O(1)$

C++ Code

```
// Insert at End
void insertAtEnd(int arr[], int &size, int capacity, int value)
{
    if (size == capacity)
    {
        cout << "Array is Full! Insertion not possible.\n";
        return;
    }
    // Place new element at index = size (end)
    arr[size] = value;
    // Increase size by 1
    size++;
}
```

Example Call

```
int arr[10] = {10, 20, 30, 40, 50};
int size = 5;
int capacity = 10;
insertAtEnd(arr, size, capacity, 60); // size becomes 6
```


Insertion – Insert at Middle

EXAMPLE

We have an array of capacity **10** and current size **5**.
We will insert **25** at index position **2**.

Initial Array

Index:	0	1	2	3	4	5	6	7	8	9
Value:	10	20	30	40	50					

size = 5

capacity = 10

Step 2: Shift elements to the right (from last index to position)

We need to create space at index 2.

Before Shifting:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	10	20	30	40	50					

Shift elements
one position
to the right

After Shifting:

Index:	0	1	2	4	5	6	7	8	9
Value:	10	20		30	40	50			

Empty space
created at
index 2

Step 3: Insert new element at index 2 and increase size

After Insertion:

Index:	0	1	2	3	4	5	6	7	8	9
Value:	10	20	25	30	40	50				

size = 6

capacity = 10

Insertion – Insert at Middle (C++ Code)

C++ CODE

```
// Insert an element at any index position
void insertAtPosition(int arr[], int &size, int capacity,
                     int position, int value)
{
    if (size == capacity)
    {
        cout << "Array is Full! Insertion not possible.\n";
        return;
    }

    // Step 1: Shift elements to the right
    for (int i = size; i > position; i--)
        arr[i] = arr[i - 1];

    // Step 2: Insert new element
    arr[position] = value;

    // Step 3: Increase size
    size++;
}
```

Example Call

```
int arr[10] = {10, 20, 30, 40, 50};
int size = 5;
int capacity = 10;
insertAtPosition(arr, size, capacity, 2, 25); // size becomes 6
```

Best Case (position = n)

- No shifting (only insertion).
- Time Complexity: **O(1)**

Worst Case (position = 0)

- Shift all n elements.
- Time Complexity: **O(n)**

Deletion – Delete at Beginning

Example: Delete the first element (10) from the array.

1. ORIGINAL ARRAY

Index	0	1	2	3	4
Array	10	20	30	40	50

Size = 5

Step:

We want to delete the element at index 0.

2. SHIFT ELEMENTS TO THE LEFT

Index	0	1	2	3	4
Array	20	30	40	50	-

Move each element one position to the left. The last element is now duplicate / garbage (will be removed).

3. DECREASE SIZE

Index	0	1	2	3	4
Array	20	30	40	50	-

Size = 4

Result:

Element 10 is deleted.
New size = 4

Algorithm (Delete at Start):

- For $i = 0$ to $\text{size} - 2$
 $\text{arr}[i] = \text{arr}[i + 1]$; // shift left
- $\text{size} = \text{size} - 1$; // decrease the size

C++ Code:

```
for(int i = 0; i < size - 1; i++)  
    arr[i] = arr[i + 1];  
size--;
```

Time Complexity:

$O(n)$

Deletion – Delete at Beginning (C++ Code)

C++ CODE

```
// Delete element at start (index 0)
void deleteAtStart(int arr[], int &size)
{
    // If array is already empty
    if (size == 0)
    {
        cout << "Array is Empty! Deletion not possible.\n";
        return;
    }

    // Step 1: Shift elements one position to the left
    for (int i = 0; i < size - 1; i++)
        arr[i] = arr[i + 1];

    // Step 2: Decrease size by 1
    size--;
}
```

Example Call

```
int arr[10] = {10, 20, 30, 40, 50};
int size = 5;
deleteAtStart(arr, size);    // size becomes 4
```


Deletion – Delete at End

Example: Delete the last element (50) from the array.

1. Original Array

Index	0	1	2	3	4
Array	10	20	30	40	50

Size = 5

Step:

We want to delete the last element at index 4.

2. Remove Last Element

Index	0	1	2	3	
Array	10	20	30	40	-

Last element is removed

What Happens?

- No shifting of elements is required.
- We simply reduce the size by 1.
- The last element is considered deleted.

3. Resultant Array

Index	0	1	2	3
Array	10	20	30	40

Size = 4

Time Complexity:

$O(1)$

(Constant time – only size is reduced, no elements are shifted)

Deletion – Delete at End (C++ Code)

C++ CODE

```
// Delete element at end (last index)
void deleteAtEnd(int arr[], int &size)
{
    if (size == 0)
    {
        cout << "Array is Empty! Deletion not possible.\n";
        return;
    }
    // Simply decrease size by 1
    size--;
}
```

Example Call

```
int arr[10] = {10, 20, 30, 40, 50};
int size = 5;
deleteAtEnd(arr, size); // size becomes 4
```


Deletion – Delete at Middle (Specific Index)

Example: Delete the element 30 at index 2 from the array.

1. ORIGINAL ARRAY

Index	0	1	2	3	4
Array	10	20	30	40	50

↑
Element to be deleted

Size = 5

Step:

We want to delete the element at index 2 (value = 30).

2. SHIFT ELEMENTS TO THE LEFT

Index	0	1	2	3	4
Array	10	20	40	50	×

Diagram illustrating the shift of elements to the left. Red arrows show the element at index 3 (40) moving to index 2, and the element at index 4 (50) moving to index 3. The element at index 2 (30) is marked for deletion.

What happens?

- Move each element after index 2 one position to the left.
- The last element becomes duplicate / garbage.

3. DECREASE SIZE

Index	0	1	2	3	4
Array	10	20	40	50	–

Size = 4

Result:

- Element 30 is deleted.
- New size = 4

Deletion – Delete at Middle (Specific Index) C++ Code

C++ CODE

```
// Delete element at specific index (position)
void deleteAtPosition(int arr[], int &size, int position)
{
    // Validate position
    if (position < 0 || position >= size)
    {
        cout << "Invalid Position!\n";
        return;
    }

    // Step 1: Shift elements to the left
    for (int i = position; i < size - 1; i++)
        arr[i] = arr[i + 1];

    // Step 2: Decrease size
    size--;
}
```

Example Call

```
int arr[10] = {10, 20, 30, 40, 50};
int size = 5;
deleteAtPosition(arr, size, 2); // deletes 30, size becomes 4
```

Deleting at a specific index may require shifting up to $(n - \text{index} - 1)$ elements. In worst case, it is $O(n)$.

Search Operation – Linear Search

Problem: Search the element **40** in the array.

ARRAY

Index	0	1	2	3	4
Elements	10	20	30	40	50

Size = 5

What are we doing?

We compare the search key (**40**) with each element from left to right until we find it or reach the end.

Search Key

Key = **40**

STEP-BY-STEP PROCESS

Step 1 Compare key **40** with element at index 0

10	20	30	40	50
----	----	----	----	----



Not Equal
(Move to next index)

Step 2 Compare key **40** with element at index 1

10	20	30	40	50
----	----	----	----	----



Not Equal
(Move to next index)

Step 3 Compare key **40** with element at index 2

10	20	30	40	50
----	----	----	----	----



Not Equal
(Move to next index)

Step 4 Compare key **40** with element at index 3

10	20	30	40	50
----	----	----	----	----



Equal!
Element found at index 3.
Search successful.



Key Idea

We check each element one by one from the beginning. If we find the key, we return its index. If we reach the end without finding it, the key is not present.

Possible Outcomes

- Element **found** → Return its index.
- Element **not found** → Return -1 (or Not Found).

TIME COMPLEXITY

- Best Case : $O(1)$
(Element is at the first index)
- Average Case : $O(n)$
- Worst Case : $O(n)$
(Element is at the last index or not present)

where n = size of the array

Search Operation – Linear Search (C++ Code)

C++ CODE

```
// Linear Search: Returns index if found, otherwise -1
int linearSearch(int arr[], int size, int key)
{
    // Traverse the array
    for (int i = 0; i < size; i++)
    {
        if (arr[i] == key)
            return i;    // Key found, return index
    }
    // Key not found
    return -1;
}
```

Example Call

```
int arr[] = {15, 23, 7, 42, 8, 30};
int size = 6;
int key = 42;
int index = linearSearch(arr, size, key);
// index will be 3
// If key = 99, index will be -1
```


Update Operation at Specific Index

EXAMPLE

Array (size = 6)

Index :	0	1	2	3	4	5
Value :	15	23	7	42	8	30

We will update the value at index **3** from **42** to **99**.

HOW IT WORKS

Step 1: Check if the index is valid

Index = 3, size = 6 $\rightarrow 0 \leq 3 < 6 \rightarrow$ Valid index.

Step 2: Update the value at index 3

Before Update:

Index :	0	1	2	3	4	5
Value :	15	23	7	42	8	30

↑
Old Value = 42

After Update ($\text{arr}[3] = 99$):

Index :	0	1	2	3	4	5
Value :	15	23	7	99	8	30

↑
New Value = 99



Only one element is changed. No shifting is required.

Update Operation at Specific Index – C++ Code

C++ CODE

```
// Update element at a specific index
bool updateAtIndex(int arr[], int size, int index, int newValue)
{
    // Check if index is valid
    if (index < 0 || index >= size)
    {
        cout << "Invalid Index! Update not possible.\n";
        return false;
    }
    // Update the value
    arr[index] = newValue;
    return true;
}
```

Example Call

```
int arr[10] = {15, 23, 7, 42, 8, 30};
int size = 6;
int index = 3;
int newValue = 99;
updateAtIndex(arr, size, index, newValue); // arr[3] becomes 99
```

Update Operation at Specific Index – Complexity Analysis

COMPLEXITY ANALYSIS

Case	Time Complexity	Explanation
Best Case	<u>$O(1)$</u>	Index is valid. One assignment is done.
Worst Case	<u>$O(1)$</u>	Index is invalid. Only a comparison is done.
Average Case	<u>$O(1)$</u>	On average, constant time is taken.